# MANUAL DE REQUERIMIENTOS

## Sistema de Joyería en Línea

### ✦ LuxGem Jewelry Store ✦

Proyecto Colaborativo — Versión 1.0

Fecha: Febrero 2026

*Stack: React (Next.js) · Node.js (Express) · PostgreSQL · Docker*

# 1. Descripción General del Proyecto

LuxGem es un sistema web de joyería en línea construido con una arquitectura de tres capas: frontend, backend y base de datos. El proyecto tiene como objetivo proporcionar una plataforma completa para la gestión y venta de joyas, incluyendo catálogo de productos, carrito de compras, gestión de clientes y panel de administración.

El sistema está diseñado para trabajo colaborativo en equipo, utilizando Docker para garantizar entornos de desarrollo consistentes y GitHub para el control de versiones.

## 1.1 Objetivos del Proyecto

- Construir una aplicación web full-stack funcional con arquitectura separada (frontend/backend/BD).
- Implementar contenedores Docker para cada capa del sistema.
- Publicar imágenes en DockerHub para evaluación del docente.
- Colaborar eficientemente mediante GitHub con ramas por equipo/función.
- Implementar las 6 funciones principales del sistema de joyería.

## 1.2 Equipo y Roles Sugeridos

| Rol | Responsabilidad Principal | Tecnología |
|---|---|---|
| Dev Frontend 1 | Catálogo, página principal, UI/UX | Next.js / React |
| Dev Frontend 2 | Carrito, checkout, autenticación UI | Next.js / React |
| Dev Backend 1 | API REST, autenticación, productos | Node.js / Express |
| Dev Backend 2 | Órdenes, pagos, reportes | Node.js / Express |
| Dev BD / DevOps | Esquema DB, Docker, Docker Hub, CI | PostgreSQL / Docker |

# 2. Stack Tecnológico

A continuación se detalla el stack tecnológico seleccionado para cada capa del sistema, con justificación de su elección.

## 2.1 Frontend — Next.js (React)

Se elige Next.js como framework principal del frontend por las siguientes razones:

- Renderizado del lado del servidor (SSR) y generación estática (SSG) para mejor SEO en el catálogo de joyas.
- Sistema de rutas basado en archivos, simple de mantener en equipo.
- Soporte nativo para API Routes (opcional) y optimización de imágenes de productos.
- Compatible con Tailwind CSS para estilos rápidos y consistentes.

| Paquete / Tool | Uso |
|---|---|
| next@14 | Framework principal con App Router |
| react@18 | Librería UI base |
| tailwindcss | Estilos utilitarios |
| axios / fetch | Llamadas a la API REST del backend |
| react-query | Manejo de estado del servidor y caché |
| zustand | Estado global del carrito de compras |
| next-auth | Autenticación con sesiones JWT |

## 2.2 Backend — Node.js con Express

Node.js con Express es el framework elegido para el backend REST API por su simplicidad, ecosistema amplio y compatibilidad ideal con proyectos académicos colaborativos.

- Express proporciona un servidor HTTP minimalista y fácil de escalar con middlewares.
- Integración fluida con PostgreSQL via pg o Sequelize ORM.
- Facilita la implementación de JWT para autenticación stateless.

| Paquete / Tool | Uso |
|---|---|
| express@4 | Framework REST API |
| pg / sequelize | Conexión y ORM para PostgreSQL |
| jsonwebtoken | Generación y validación de tokens JWT |
| bcryptjs | Hashing de contraseñas |

| | |
|---|---|
| cors | Configuración de CORS entre frontend y backend |
| dotenv | Variables de entorno |
| express-validator | Validación de entradas en los endpoints |
| multer | Subida de imágenes de productos |

## 2.3 Base de Datos — PostgreSQL

PostgreSQL es un sistema gestor de base de datos relacional robusto, de código abierto, ideal para la estructura de datos de una joyería con relaciones entre productos, clientes y órdenes.

- Soporte para transacciones ACID, garantizando consistencia en órdenes de compra.
- Imagen oficial Docker disponible en DockerHub (postgres:15-alpine).
- Compatible con Sequelize ORM para migraciones y modelos.

## 2.4 Infraestructura — Docker y Docker Compose

- Cada capa del sistema se encapsula en su propio contenedor Docker.
- Docker Compose orquesta los tres contenedores localmente con una sola instrucción.
- Las imágenes se publican en DockerHub bajo el nombre del equipo para evaluación.
- Archivo .env para configurar puertos, credenciales de BD y secretos JWT.

# 3. Las 6 Funciones Principales del Sistema

El sistema LuxGem implementa seis funciones core que atraviesan las tres capas de la arquitectura. A continuación se describe cada función con sus componentes frontend, endpoints de la API y estructura de datos.

## FUNCIÓN 1 — Catálogo de Joyas

Permite a los usuarios explorar el catálogo de joyas con filtros por categoría, precio, material y búsqueda por nombre.

### Frontend (Next.js)

- Página /catalog con grid de tarjetas de productos con imagen, nombre y precio.
- Componente <FilterSidebar> con filtros por categoría (anillos, collares, pulseras), material (oro, plata, platino) y rango de precios.
- Componente <ProductCard> con imagen optimizada (next/image), nombre, precio y botón de agregar al carrito.
- Página /catalog/[id] con detalle del producto: galería de imágenes, descripción, materiales, talla y stock disponible.

### Backend (Node.js / Express)

- GET /api/products — Retorna lista paginada de productos con filtros opcionales (?category=rings&minPrice=100).
- GET /api/products/:id — Retorna detalle completo de un producto.
- GET /api/categories — Retorna lista de categorías disponibles.

### Base de Datos (PostgreSQL)

- Tabla products: id, name, description, price, stock, material, category_id, image_url, created_at.
- Tabla categories: id, name, description.

## FUNCIÓN 2 — Autenticación de Usuarios

Permite a los clientes registrarse, iniciar sesión y gestionar su sesión de forma segura mediante JWT.

### Frontend (Next.js)

- Página /register con formulario: nombre, email, contraseña, confirmación de contraseña.

- Página /login con formulario: email y contraseña.
- Uso de next-auth o manejo manual de JWT almacenado en httpOnly cookie.
- Componente <AuthGuard> para proteger rutas privadas como /profile y /orders.

### Backend (Node.js / Express)

- POST /api/auth/register — Crea nuevo usuario con contraseña hasheada (bcrypt).
- POST /api/auth/login — Valida credenciales y retorna JWT con expiración de 24h.
- GET /api/auth/me — Retorna perfil del usuario autenticado (requiere token).
- Middleware authMiddleware.js para proteger rutas privadas verificando el JWT.

### Base de Datos (PostgreSQL)

- Tabla users: id, name, email, password_hash, role (customer/admin), created_at.

## FUNCIÓN 3 — Carrito de Compras

Permite a los usuarios agregar productos al carrito, modificar cantidades y visualizar el resumen de compra.

### Frontend (Next.js)

- Estado global del carrito manejado con Zustand (persiste en localStorage).
- Componente <CartDrawer>: panel lateral deslizante con lista de items, cantidades editables y botón de checkout.
- Componente <CartIcon> en el header con badge del número de items.
- Página /cart con vista completa del carrito, resumen de precios y subtotal.

### Backend (Node.js / Express)

- El carrito se gestiona principalmente en el frontend con Zustand.
- POST /api/cart/validate — Valida que los productos del carrito tengan stock disponible antes de procesar la orden.

### Base de Datos (PostgreSQL)

- No requiere tabla propia; el carrito vive en el estado del frontend hasta confirmarse como orden.

## FUNCIÓN 4 — Gestión de Órdenes / Checkout

Permite a los usuarios autenticados procesar su compra, generar una orden y recibir confirmación.

### Frontend (Next.js)

- Página /checkout con formulario de dirección de envío, resumen del pedido y botón de confirmar orden.
- Página /orders con historial de órdenes del usuario (número, fecha, total, estado).
- Página /orders/[id] con detalle de la orden: productos, cantidades, dirección y estado de envío.

### Backend (Node.js / Express)

- POST /api/orders — Crea una nueva orden, descuenta stock y devuelve el número de orden. Requiere autenticación.
- GET /api/orders — Lista las órdenes del usuario autenticado.
- GET /api/orders/:id — Retorna detalle de una orden específica.
- PATCH /api/orders/:id/status — (Admin) Actualiza el estado de la orden (pending, processing, shipped, delivered).

### Base de Datos (PostgreSQL)

- Tabla orders: id, user_id, total, status, shipping_address, created_at.
- Tabla order_items: id, order_id, product_id, quantity, unit_price.

## FUNCIÓN 5 — Panel de Administración

Permite al administrador gestionar el inventario de joyas: agregar, editar y eliminar productos, y ver reportes de ventas.

### Frontend (Next.js)

- Ruta protegida /admin accesible solo para usuarios con role=admin.
- Página /admin/products con tabla de todos los productos: buscar, filtrar, editar, eliminar.
- Formulario /admin/products/new y /admin/products/[id]/edit para crear y editar joyas con subida de imagen.
- Página /admin/orders con lista de todas las órdenes y opción para cambiar estado.

### Backend (Node.js / Express)

- POST /api/products — Crea un nuevo producto (requiere role admin).
- PUT /api/products/:id — Actualiza datos de un producto.

- DELETE /api/products/:id — Elimina un producto del catálogo.
- POST /api/upload — Sube imagen de producto al servidor (multer) y retorna URL.
- Middleware adminMiddleware.js para verificar que el usuario tiene rol de administrador.

### Base de Datos (PostgreSQL)

- Uso de las tablas products y categories ya definidas.
- Campo role en la tabla users distingue entre customer y admin.

## FUNCIÓN 6 — Reseñas y Valoraciones

Permite a clientes que han comprado un producto dejar reseñas y calificaciones (1-5 estrellas), y ver el rating promedio.

### Frontend (Next.js)

- Componente <ReviewSection> en la página de detalle del producto /catalog/[id].
- Formulario de reseña visible solo para usuarios autenticados que han comprado el producto.
- Visualización de rating promedio con estrellas y número de reseñas.
- Lista de reseñas con nombre del usuario, fecha, calificación y comentario.

### Backend (Node.js / Express)

- GET /api/products/:id/reviews — Lista todas las reseñas de un producto con rating promedio.
- POST /api/products/:id/reviews — Crea una reseña. Verifica que el usuario haya comprado el producto.
- DELETE /api/reviews/:id — (Admin) Elimina una reseña inapropiada.

### Base de Datos (PostgreSQL)

- Tabla reviews: id, product_id, user_id, rating (1-5), comment, created_at.
- Constraint UNIQUE (product_id, user_id) para evitar reseñas duplicadas.

# 4. Estructura del Repositorio GitHub

El repositorio principal contendrá tres carpetas raíz correspondientes a cada capa del sistema, más los archivos de orquestación Docker.

```
luxgem-jewelry/
├── frontend/              ← Proyecto Next.js
│   ├── src/app/           ← Páginas (App Router)
│   ├── src/components/    ← Componentes reutilizables
│   ├── src/store/         ← Zustand (carrito)
│   ├── Dockerfile
│   └── .env.local
├── backend/               ← Proyecto Node.js / Express
│   ├── src/routes/        ← Rutas de la API
│   ├── src/controllers/   ← Lógica de negocio
│   ├── src/middleware/    ← Auth, Admin
│   ├── src/models/        ← Modelos Sequelize
│   ├── migrations/        ← Migraciones de BD
│   ├── Dockerfile
│   └── .env
├── database/              ← Scripts SQL iniciales
│   ├── init.sql           ← Creación de tablas y seed data
│   └── Dockerfile
├── docker-compose.yml    ← Orquestación de los 3 contenedores
└── README.md             ← Instrucciones de instalación y uso
```

# 5. Configuración Docker

Cada servicio tiene su propio Dockerfile y se orquestan juntos con docker-compose.yml. A continuación se presentan los archivos de configuración esenciales.

## 5.1 docker-compose.yml

```yaml
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: luxgem_db
      POSTGRES_USER: luxgem_user
      POSTGRES_PASSWORD: luxgem_pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
    ports: ['5432:5432']

  backend:
    build: ./backend
    ports: ['4000:4000']
    depends_on: [postgres]
    environment:
      DATABASE_URL: postgres://luxgem_user:luxgem_pass@postgres:5432/luxgem_db
      JWT_SECRET: your_secret_here

  frontend:
    build: ./frontend
    ports: ['3000:3000']
    depends_on: [backend]
    environment:
      NEXT_PUBLIC_API_URL: http://backend:4000/api
```

## 5.2 Publicación en DockerHub

Para subir las imágenes a DockerHub y permitir la evaluación del docente, cada integrante ejecuta los siguientes pasos:

1. docker build -t tuusuario/luxgem-frontend:1.0 ./frontend
2. docker build -t tuusuario/luxgem-backend:1.0 ./backend
3. docker login (ingresar credenciales DockerHub)

4. docker push tuusuario/luxgem-frontend:1.0
5. docker push tuusuario/luxgem-backend:1.0

El evaluador podrá descargar las imágenes con docker pull y levantar el sistema completo con docker-compose up.

# 6. Plan de Trabajo Colaborativo por Etapas

El proyecto se divide en 4 etapas ordenadas para facilitar la colaboración y evitar bloqueos entre desarrolladores.

## ETAPA 1 — Configuración Inicial (Días 1-2)

Responsable: Todo el equipo. Objetivo: tener el entorno listo para desarrollar.

6. Crear organización en GitHub y repositorio luxgem-jewelry.
7. Definir la estrategia de ramas: main, develop, feature/[nombre-función].
8. Dev BD/DevOps: crear docker-compose.yml y Dockerfiles base para los tres servicios.
9. Dev Backend: inicializar proyecto Node.js + Express con estructura de carpetas.
10. Dev Frontend: inicializar proyecto Next.js con Tailwind CSS configurado.
11. Dev BD: escribir init.sql con creación de todas las tablas (con datos de prueba/seed).
12. Verificar que docker-compose up levanta los tres contenedores sin errores.

## ETAPA 2 — Funciones Base (Días 3-7)

Responsable: Backend 1 + Frontend 1. Objetivo: catálogo y autenticación funcionando.

13. Backend 1: implementar endpoints de productos (GET /api/products, GET /api/products/:id).
14. Backend 1: implementar autenticación (POST /api/auth/register, POST /api/auth/login, middleware JWT).
15. Frontend 1: implementar página /catalog con grid de productos consumiendo la API.
16. Frontend 2: implementar páginas /login y /register con formularios y llamadas a la API.
17. Integración y prueba: el frontend puede mostrar joyas y los usuarios pueden autenticarse.

## ETAPA 3 — Funciones Principales (Días 3-7)

Responsable: Backend 2 + Frontend 2 + Frontend 1 continúa. Objetivo: carrito, órdenes y admin.

18. Backend 2: implementar endpoints de órdenes (POST, GET /api/orders).
19. Frontend 2: implementar carrito con Zustand y componente <CartDrawer>.
20. Frontend 2: implementar página /checkout y /orders.
21. Backend 1: implementar endpoints de administración (POST/PUT/DELETE /api/products).
22. Frontend 1: implementar panel de administración /admin con tabla de productos.
23. Backend 2 + Frontend 1: implementar función de reseñas (CRUD /api/products/:id/reviews).

## ETAPA 4 — Docker, Testing y Entrega (Día 8)

Responsable: Todo el equipo. Objetivo: sistema completo, dockerizado y subido a DockerHub.

24. Finalizar y revisar todos los Dockerfiles para producción.
25. Construir imágenes finales y hacer push a DockerHub con tag :latest y :1.0.
26. Hacer merge de todas las ramas a develop y luego a main.
27. Escribir README.md con instrucciones de instalación, uso y links a DockerHub.
28. Prueba final: clonar el repositorio en una máquina limpia y verificar que docker-compose up funciona.

# 7. Esquema de Base de Datos

El siguiente diagrama muestra todas las tablas del sistema y sus relaciones principales.

| Tabla | PK / FK | Columnas Principales |
|---|---|---|
| users | PK: id | name, email, password_hash, role, created_at |
| categories | PK: id | name, description |
| products | PK: id \| FK: category_id | name, description, price, stock, material, image_url, created_at |
| orders | PK: id \| FK: user_id | total, status, shipping_address, created_at |
| order_items | PK: id \| FK: order_id, product_id | quantity, unit_price |
| reviews | PK: id \| FK: product_id, user_id | rating (1-5), comment, created_at |

# 8. Estrategia de Colaboración en GitHub

Para evitar conflictos y mantener el código organizado, el equipo seguirá el flujo de trabajo Git Feature Branch:

## 8.1 Ramas del Repositorio

- main — Rama de producción. Solo recibe merges desde develop una vez aprobado.
- develop — Rama de integración. Aquí se fusionan todas las features terminadas.
- feature/catalog — Catálogo de joyas (Dev Frontend 1).
- feature/auth — Autenticación (Dev Backend 1 + Dev Frontend 2).
- feature/cart-orders — Carrito y órdenes (Dev Frontend 2 + Dev Backend 2).
- feature/admin — Panel de administración (Dev Backend 1 + Dev Frontend 1).
- feature/reviews — Reseñas y valoraciones (Dev Backend 2 + Dev Frontend 1).
- feature/docker — Configuración Docker y CI (Dev BD/DevOps).

## 8.2 Flujo de Trabajo

29. Crear la rama: git checkout -b feature/mi-función
30. Desarrollar y hacer commits descriptivos: git commit -m 'feat: agregar endpoint GET /api/products'
31. Subir la rama: git push origin feature/mi-función
32. Abrir un Pull Request hacia develop con descripción de los cambios.

33. Al menos un compañero debe revisar y aprobar el PR antes de hacer merge.
34. Resolver conflictos en la rama feature antes de hacer merge.

## 9. Checklist de Entrega

Antes de la fecha de evaluación, verificar que todos los siguientes ítems están completos:

| # | Ítem de Entrega | Responsable |
|---|---|---|
| 1 | Repositorio GitHub público con README completo | Todo el equipo |
| 2 | docker-compose up levanta los 3 servicios sin errores | DevOps |
| 3 | Imágenes publicadas en DockerHub (frontend + backend) | DevOps |
| 4 | Función 1: Catálogo funcionando con filtros | Frontend 1 + Backend 1 |
| 5 | Función 2: Registro e inicio de sesión con JWT | Backend 1 + Frontend 2 |
| 6 | Función 3: Carrito de compras funcional | Frontend 2 |
| 7 | Función 4: Checkout y historial de órdenes | Backend 2 + Frontend 2 |
| 8 | Función 5: Panel de administración para CRUD productos | Backend 1 + Frontend 1 |
| 9 | Función 6: Sistema de reseñas y valoraciones | Backend 2 + Frontend 1 |
| 10 | Base de datos con tablas, relaciones y datos de prueba | Dev BD |